

100 DOM Challenges (From Basics to Advanced)

■ Beginner (Level 1 – Basics) (1–20)

1. Select a `<p>` tag and change its text.
2. Change the color of a `<h1>` tag to red using JavaScript.
3. Select an element by id and change its background color.
4. Select multiple elements by class and change their text one by one.
5. Change the text content of a button when it is clicked.
6. Select an `<input>` element and log its value to the console.
7. Create a function that changes the page title dynamically.
8. Change the src of an `<img>` element when a button is clicked.
9. Change the font size of a paragraph dynamically with JavaScript.
10. Change the style of a specific element only (without affecting others).
11. Hide an element when a button is clicked.
12. Show a hidden element when a button is clicked.
13. Toggle the visibility of an element on button click.
14. Change the text of multiple `<li>` elements inside a `<ul>`.
15. Add inline CSS style using `.style` property.
16. Change the innerHTML of a `<div>` dynamically.
17. Log the tag name of a clicked element.
18. Change the entire background color of the page.
19. Replace one element's content with another element's content.
20. Add a new `<p>` element dynamically to the page.

■ Intermediate (Level 2 – Events & Attributes) (21–50)

21. Change text color when hovering over an element.
22. Add a click event to multiple buttons using `querySelectorAll`.
23. Add a new CSS class to an element when clicked.
24. Remove a CSS class from an element when double-clicked.
25. Toggle a class every time a button is clicked.
26. Read the href value of a link and log it.
27. Change the href of a link dynamically.
28. Disable a button after clicking it once.
29. Enable a disabled button on another button click.
30. Create an event that changes the image on mouse hover.
31. Add a keydown event that changes background color when you press a key.
32. Show typed text from an input field live inside a `<p>` tag.
33. Change the border color of an input when it is focused.
34. Prevent a link from opening using `event.preventDefault()`.
35. Stop event bubbling in nested elements using `stopPropagation()`.
36. Create a function that counts how many times a button is clicked.
37. Change one element when another element is clicked (e.g., click button → change heading).
38. Read and modify a `data-*` attribute.
39. Log all child elements of a parent element.
40. Get and display the parent of a clicked element.
41. Select the first and last child of a list and style them differently.
42. Dynamically add a class to every even list item.
43. Dynamically add a class to every odd list item.
44. Remove a specific child element from a parent using `.removeChild()`.
45. Remove an element directly using `.remove()`.

46. Replace an existing element with a new one dynamically.
47. Clone an element and add it somewhere else on the page.
48. Create a button that duplicates a paragraph when clicked.
49. Move an element from one container to another.
50. Detect which element was clicked using event.target.

■ Advanced (Level 3 – Dynamic DOM Manipulation) (51–80)

51. Dynamically create a and add it to an existing .
52. Create multiple elements dynamically from an array and append them.
53. Create a form with JavaScript and append it to the DOM.
54. Create a live character counter for a textarea.
55. Show an error message below an input when it's empty on submit.
56. Remove all children of a container with one click.
57. Create a table dynamically using JavaScript.
58. Add a new row to a table on button click.
59. Highlight a table row when clicked.
60. Create a dropdown list dynamically using JavaScript.
61. Update dropdown options based on user selection (cascading dropdown).
62. Create an image gallery that switches images when thumbnails are clicked.
63. Create a counter with increment and decrement buttons.
64. Make a simple to-do list where items can be added and removed.
65. Add a delete button next to each list item to remove it individually.
66. Add edit functionality to list items.
67. Create a modal popup that shows on button click.
68. Close the modal when clicking outside it.
69. Create a simple accordion (expand/collapse content).
70. Make a collapsible navigation menu.
71. Create a tooltip that appears when hovering over an element.
72. Create a simple progress bar and update it dynamically.
73. Animate the progress bar filling using setInterval.
74. Create a slideshow that changes images every few seconds.
75. Pause the slideshow when hovering over the image.
76. Resume the slideshow when mouse leaves the image.
77. Create tabs (switch content when tab buttons are clicked).
78. Make a dark mode/light mode toggle using DOM manipulation.
79. Store selected theme (dark/light) in localStorage and apply on reload.
80. Detect window resizing and update element size dynamically.

■ Expert (Level 4 – Real-World, Complex UIs) (81–100)

81. Create a search filter that hides list items not matching the query.
82. Create a real-time word counter that updates as you type.
83. Create a table sorter (sort rows by clicking column headers).
84. Create a live clock using setInterval.
85. Create a countdown timer with start/pause/reset buttons.
86. Build a dynamic cart UI (add/remove products with total price update).
87. Build a form validator that checks multiple fields dynamically.
88. Show form error messages in real time as user types.
89. Create a scroll-to-top button that appears when scrolling down.
90. Implement infinite scrolling (load more content when reaching bottom).
91. Create a sticky header that stays on top while scrolling.
92. Lazy load images when they come into view.
93. Highlight the current section in the navbar while scrolling (scroll spy).
94. Build a simple drag-and-drop list reorder feature.

95. Build a mini color picker that changes page background.
96. Implement a virtual list (render only visible items in a large list).
97. Debounce user input before running a search function.
98. Throttle scroll events to avoid performance issues.
99. Create a mini DOM diff checker (compare two DOM trees and highlight changes).
100. Build a mini React-like component system (create elements via functions, re-render on state change).